



Tiny-GPT

Este modelo é baseado na arquitetura **Transformer**, inspirado nos modelos do tipo **GPT (Generative Pre-trained Transformer)**.

Ao longo deste Colab, iremos navegar pelas componentes cruciais dos modelos GPT-like, que atualmente dominam grande parte do mundo da Inteligência Artificial.

No final, teremos um modelo funcional no estilo GPT, capaz de realizar inferência.

No entanto, é importante ter em conta que modelos desta dimensão apresentam limitações na geração de texto com consistência lexical.

Este Colab é extensão de um outro colab [Transformer - Attention Is All You Need](#) que faz parte do rep [Tiny-GPT](#).

Todas as diferenças entre ambos os colabs estarão marcadas com uma ★.

```
In [57]: !wget https://raw.githubusercontent.com/Eliezer-Carvalho/Tiny-GPT/refs/heads/main/data/0s%20Lusiadas.txt
--2026-05-17 17:49:39-- https://raw.githubusercontent.com/Eliezer-Carvalho/Tiny-GPT/refs/heads/main/data/0s%20Lusiadas.txt
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 321644 (314K) [text/plain]
Saving to: '0s Lusiadas.txt'

0s Lusiadas.txt      100%[=====>] 314.11K  --.-KB/s    in 0.02s

2026-05-17 17:49:40 (14.8 MB/s) - '0s Lusiadas.txt' saved [321644/321644]
```

Libraries

```
In [56]: import torch
import torch.nn as nn
import torch.nn.functional as F
import math
import matplotlib.pyplot as plt

device = "cuda" if torch.cuda.is_available() else "cpu" #Modelos LLM rodam em
print (device)
```

cpu

Estatísticas do Dataset

```
In [58]: import re      #https://pypi.org/project/rexex/

with open ("Os Lusíadas.txt", "r", encoding = "utf-8") as f:
    dataset = f.read()

#https://regexbox.com/cheatsheet
print (f"Caractéres --> {len(dataset)}")
print (f"Palavras --> {len(re.findall(r'\w+', dataset))}")
print (f"Linhas --> {len(re.findall(r'\n', dataset))}")
```

Caractéres --> 314636

Palavras --> 55825

Linhas --> 9940

Char Level Tokenizer

No repositório que acompanha este projeto, [Transformer - Attention Is All You Need](#), foi realizado um estudo de diferentes Tokenizers num dataset distinto, tanto em termos de linguagem quanto de tamanho.

O dataset utilizado neste repositório atual é relativamente pequeno (**315KB**). A partir do estudo acima mencionado e alguns testes com modelos treinados, concluiu-se que algoritmos tradicionais de tokenização, como **BPE** ou **Unigram**, não são ideais para datasets pequenos.

Isto ocorre porque os modelos Transformers necessitam de grandes quantidades de tokens para gerar texto de forma **lexical e semanticamente consistente**.

Para se ter um exemplo, o primeiro modelo GPT lançado em 2018 juntamente com o paper [Improving Language Understanding by Generative Pre-Training](#) foi treinado com aproximadamente **5 bilhões de Tokens!!**

Tendo um dataset pequeno, a ideia é maximizar ao máximo o nível de Tokens.

Para tal, aplicamos um Character-Level Tokenizer

```
In [59]: #https://www.youtube.com/watch?v=kCc8FmEb1nY&t=6141s #Código proveniente deste

chars = sorted(list(set(dataset))) #Vocab_Size #Número de Tokens Únicos

numerate = {char:i for i, char in enumerate(chars)}
itos = {i:char for i, char in enumerate(chars)}

encode = lambda s: [numerate[c] for c in s]
decode = lambda l: ''.join([itos[i] for i in l])
```

```
print (f"Vocab_Size -> {len(chars)}")
print (f"Encoder -> {encode("Os Lusíadas são fixes!")}")
print (f"Decoder -> {decode(encode("Os Lusíadas são fixes!"))}")
```

Vocab_Size -> 85

Encoder -> [26, 55, 1, 23, 57, 55, 46, 38, 41, 38, 55, 1, 55, 72, 51, 1, 43, 46, 59, 42, 55, 2]

Decoder -> Os Lusíadas são fixes!

Tokenização

```
In [60]: dataset_tokenizado = encode(dataset)

print (dataset [36:103], "\n")
print (dataset_tokenizado [36:103], "\n")

print (f"Tiny-GPT é alimentado por {len(dataset_tokenizado)} Tokens!")
```

Que da ocidental praia Lusitana,
Por mares nunca de antes navegados

[28, 57, 42, 1, 41, 38, 1, 51, 40, 46, 41, 42, 50, 56, 38, 48, 1, 52, 54, 38, 46, 38, 1, 23, 57, 55, 46, 56, 38, 50, 38, 7, 0, 27, 51, 54, 1, 49, 38, 54, 42, 55, 1, 50, 57, 50, 40, 38, 1, 41, 42, 1, 38, 50, 56, 42, 55, 1, 50, 38, 58, 42, 44, 38, 41, 51, 55]

Tiny-GPT é alimentado por 314636 Tokens!

Tensor

```
In [61]: tensor = torch.tensor (dataset_tokenizado, dtype = torch.long) #https://docs.pytorch.org/docs/tensors.html

print (tensor.dtype)
print (tensor.shape)
print (tensor [:100])
```

torch.int64

torch.Size([314636])

tensor([0, 0, 13, 55, 1, 38, 54, 49, 38, 55, 1, 42, 1, 51, 55, 1, 39, 38, 54, 81, 42, 55, 1, 38, 55, 55, 46, 50, 38, 48, 38, 41, 51, 55, 7, 0, 28, 57, 42, 1, 41, 38, 1, 51, 40, 46, 41, 42, 50, 56, 38, 48, 1, 52, 54, 38, 46, 38, 1, 23, 57, 55, 46, 56, 38, 50, 38, 7, 0, 27, 51, 54, 1, 49, 38, 54, 42, 55, 1, 50, 57, 50, 40, 38, 1, 41, 42, 1, 38, 50, 56, 42, 55, 1, 50, 38, 58, 42, 44, 38])

Dados de Treino e Dados de Validação

Atenção a esta fase

Importante para mais tarde treinar e avaliar o modelo!

```
In [62]: data = int (0.9 * (len(tensor)))  
train = tensor [:data] #90% Dados de Treino  
val = tensor [data:] #10% Dados de Validação ou Teste
```

Batch e Sequence Length

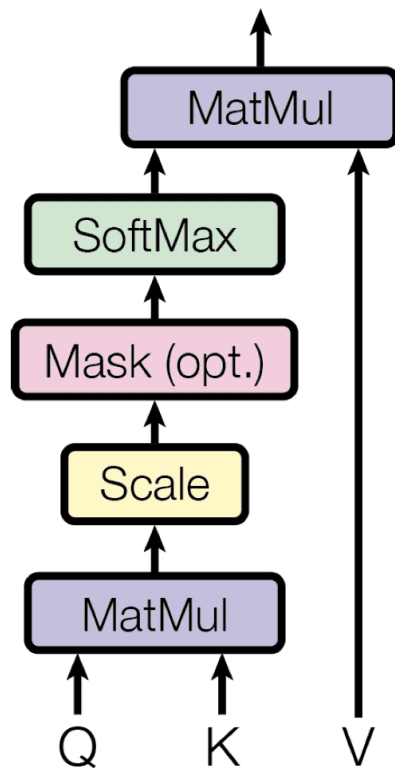
Importante, pois o modelo não recebe todos os Tokens de uma vez só, vai recebendo excertos.

```
In [63]: torch.manual_seed (2000) #Número random para determinar uma posição random  
  
def Batch (datatype):  
  
    dados = train if datatype == 'train' else val #Posteriormente no treino vai  
  
    indice = torch.randint (0, len(dados) - Sequence_Length, (Batch_Size, ))  
    train_data = torch.stack ([dados [i: i + Sequence_Length] for i in indice])  
    val_data = torch.stack ([dados [i + 1: i + Sequence_Length + 1] for i in indice])  
  
    train_data, val_data = train_data.to(device), val_data.to(device) #Passar os  
  
    return train_data, val_data
```

Definição de uma Head

Uma Head não é nada mais nada menos do que a aplicação do mecanismo **Attention**.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



A **Head Size (tamanho da cabeça)** representa o profundidade que uma Single Head irá ter.

Ou seja, cada head não “vê” a sequência inteira do embedding diretamente. Transforma os embeddings para um espaço mais pequeno (**head_size**), onde aprende relações diferentes.

Desta maneira cada head aprende padrões diferentes (ex: sintático, semântico, posição, etc.)

$$\text{head_size} = \frac{d_{\text{emb}}}{\text{heads}}$$

```

In [64]: class Head (nn.Module): #Head = Self Masked Attention
    def __init__(self):
        super().__init__()

        self.Query = nn.Linear (Embedding_Dimension, Head_Size, bias = False) #Isto
        self.Key = nn.Linear (Embedding_Dimension, Head_Size, bias = False) #Isto
        self.Value = nn.Linear (Embedding_Dimension, Head_Size, bias = False) #Isto

        self.register_buffer('tril', torch.tril(torch.ones(Sequence_Length, Sequen
        #Melhor assim, ao não criar sempre mask, a mask deixa de ser um parâmetro
        #register_buffer faz isto "Guarda este tensor dentro do modelo, mas ele NÃO
        #tril = triangular lower

        self.Dropout = nn.Dropout (0.2) #Dropout é importante para evitar overfitt
  
```

```

def forward (self, x):

    B, T, C = x.shape

    Q = self.Query (x)
    K = self.Key (x)
    V = self.Value (x)

    # Scaled Dot-Product Attention
    attention_score = Q @ K.transpose (-2, -1) * (Head_Size ** -0.5) #Raiz de

    #Self Masked Attention
    attention_score = attention_score.masked_fill (self.tril [:T, :T] == 0, fl

    #Softmax converte em probs
    attention_score = F.softmax (attention_score, dim = -1) #Normalização da a

    #Dropout para evitar overfitting
    attention_score = self.Dropout (attention_score) #Evitar overfitting

    head_output = attention_score @ V
    return head_output

```

Definição de Multi Head ★

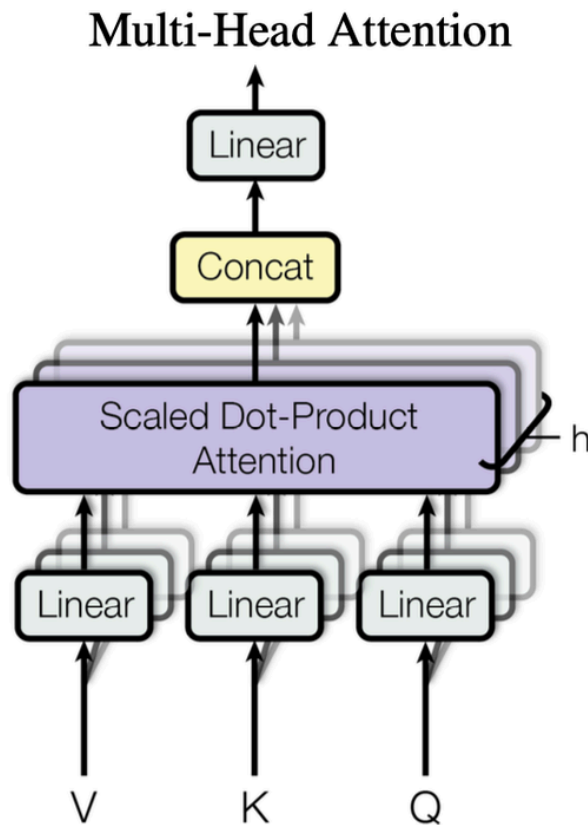
É um dos conceitos mais simples e importantes dentro da arquitetura GPT.

Consiste em concatenar os outputs de várias **heads (cabeças de atenção)** e, em seguida, aplicar uma **output projection** para combinar e misturar a informação obtida.

As **Multi-Heads** permitem que o modelo capte diferentes tipos de relações dentro de uma sequência (**Sequence Length**), analisando-a sob múltiplas perspectivas em simultâneo.

$$W^O \{ \text{MultiHead} \} (Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$



```
In [65]: class Multi_Head (nn.Module):
def __init__(self):
    super().__init__()

    self.Concat = nn.ModuleList ([Head() for i in range (Number_Head)]) #Head
    self.Output_Projection = nn.Linear (Embedding_Dimension, Embedding_Dimension)
    self.Dropout = nn.Dropout (0.2) #Dropout é importante para evitar overfitting

    def forward (self, x):

        output = self.Output_Projection (torch.cat ([f (x) for f in self.Concat],
        multi_head_output = self.Dropout(output)
        return multi_head_output
```

Definição de um Block (Pre-LayerNorm Transformer) ★

Com o paper “*Attention Is All You Need*”, foi introduzida a arquitetura Transformer, originalmente com uma abordagem chamada **Post-Norm**.

No entanto, rapidamente os investigadores identificaram problemas ao escalar

modelos mais profundos, como:

- **Treino instável em redes profundas**
- **Gradientes a explodir ou a desaparecer**

Isto levou ao surgimento da abordagem **Pre-Norm**, que melhorou significativamente a estabilidade do treino.

Post-Norm vs Pre-Norm ?

- **Abordagem Post-Norm:**

Attention -> Add -> Norm -> Feed Forward -> Add -> Norm

- **Abordagem Pre-Norm:**

Norm -> Attention -> Add -> Norm -> Feed Forward -> Add

Nos modelos atuais, o bloco típico é **Pre-Norm**, e a ordem correta é:

- Layer Norm
- Multi-Head Self-Attention
- Residual Connection (Add)
- Layer Norm
- Feed Forward Neural Network (MLP)
- Residual Connection (Add)

Estes métodos conjugados é aquilo a que chamamos de **Block** ou **Layer**.

```
In [66]: class Block (nn.Module):
def __init__(self):
    super().__init__()

    self.LayerNorm_Attention = nn.LayerNorm (Embedding_Dimension) #Layer Norm
    self.Masked_Multi_Head_Attention = Multi_Head() #Self Masked Attention

    self.LayerNorm_FNN = nn.LayerNorm (Embedding_Dimension) #Layer Norm antes
    self.FeedForward_NeuralNet = nn.Sequential ( #MLP
        nn.Linear (Embedding_Dimension, 4 * Embedding_Dimension), #*4 está no
        nn.GELU(),
        nn.Linear (Embedding_Dimension * 4, Embedding_Dimension),
        nn.Dropout (0.2) #Dropout é importante para evitar overfitting
    )
```



```

def forward (self, x):
    Norm_Attention = self.Masked_Multi_Head_Attention (self.LayerNorm_Attention(x))
    Add_Attention = x + Norm_Attention #Add #Residual Connection #Muito Importante
    x = Add_Attention

    Norm_FNN = self.FeedForward_NeuralNet (self.LayerNorm_FNN (x)) #Norm #Layer
    Add_FNN = x + Norm_FNN #Add #Residual Connection #Muito Importante!!!
    x = Add_FNN

    return x

```

Modelo Tiny-GPT ★

Implementação do Modelo Final

```

In [67]: class TINY_GPT (nn.Module):
    def __init__(self):
        super().__init__()

        self.Embedding = nn.Embedding (len(chars), Embedding_Dimension) #Embedding
        self.Positional_Encoding = nn.Embedding (Sequence_Length, Embedding_Dimension) #Positional Encoding

        self.Blocks = nn.Sequential ( #Não é a maneira ideal de se fazer. #Atenção
            Block(),
            Block(),
            Block(),
            Block(),
        )

        self.LayerNorm = nn.LayerNorm(Embedding_Dimension) #Layer Norm antes do La
        self.Language_Modeling = nn.Linear (Embedding_Dimension, len(chars)) #Conv

    def forward (self, x):
        B, T = x.shape

        x = self.Embedding (x) + self.Positional_Encoding (torch.arange (T, device=x.device))

        x = self.Blocks(x)

        x = self.LayerNorm (x)

        logits = self.Language_Modeling(x)

        return logits #Return de um tensor (B, T, VOCAB_SIZE)

```

Treino do Modelo (Super-Importante) ★

Como já vimos no código, os pesos são iniciados de forma aleatória.

Assim, é essencial um **processo de treino**, que permita atualizar estes parâmetros iterativamente com base no erro do modelo, tornando-o progressivamente mais preciso.

Para treinar estes parâmetros, é necessário um processo de otimização baseado em **gradientes**.

O modelo faz previsões e compara-as com os valores reais através de uma função de perda (*loss function*).

Uma das funções de perda mais utilizadas em problemas de linguagem é a **Cross Entropy Loss**, que penaliza o modelo quando atribui baixa probabilidade à resposta correta.

Em tarefas de linguagem, isto é essencial porque o objetivo é maximizar a probabilidade do próximo token correto.

Depois de calcular a *loss*, utilizamos o processo de backpropagation para determinar os **Gradientes**.

Os Gradientes representam a direção e a magnitude com que cada peso da rede deve ser ajustado para minimizar o erro.

Em termos simples, indicam como cada parâmetro influencia o resultado final e como deve ser alterado para melhorar a performance do modelo.

Os gradientes por si só não atualizam os pesos!

Essa tarefa é realizada pelos **Otimizadores**, que utilizam os gradientes para aplicar correções aos parâmetros de forma eficiente e estável.

Em vez de simplesmente ajustar os pesos diretamente com base no erro, os otimizadores definem a melhor forma de os atualizar ao longo do tempo, tendo em conta fatores como a taxa de aprendizagem e o comportamento histórico dos gradientes.

Entre os otimizadores mais utilizados estão o SGD (*Stochastic Gradient Descent*), o Adam e o AdamW.

Resumo:

O treino de uma arquitetura Transformer segue este ciclo:

- Forward pass (previsão do modelo)
- Cálculo da Cross Entropy Loss
- Backpropagation (cálculo dos gradientes)
- Otimizador atualiza os pesos
- Repetição até o modelo convergir

Função que Treina o modelo

```
In [68]: def Training (modelo, steps):

    train_losses = []
    val_losses = []

    modelo.train() #Ativa o treino #https://docs.pytorch.org/tutorials/beginner/

    for i in range (steps):

        x, y = Batch('train')  #(B, T) #Dados de Treino

        logits = modelo (x) #Passa os dados x pelo modelo
        #print (logits.shape)
        B, T, V = logits.shape  #(Batch, Sequence Length, Vocab_Size)

        #Cross Entropy vai determinar a diferença entre o token escolhido (x) e o
        loss = loss_function (logits.view (B * T, V), y.view (B * T)) #Entrada que
        #0 que a loss function faz é como se fosse um aluno que responde a uma per

        optimizer.zero_grad() #Zera os gradientes acumulados nos parâmetros. #Neces
        loss.backward() #Após calcular o erro, o backprop passa pelos pesos todos

        optimizer.step() #Optimizer aplicado #0 optimizer é o que muda o valor dos

        if i % 1000 == 0: #Printa de 1000 em 1000

            losses = Calc_Loss (modelo)

            train_losses.append(losses["train"])
            val_losses.append(losses["val"])

            print(f"Step {i} | Train Loss: {losses['train']:.4f} | Validation Loss:

    return train_losses, val_losses
```

Calcular a Loss

Esta função foi altamente influenciada por este rep -> [nanoGPT](#)

```
In [69]: https://github.com/karpathy/nanoGPT/blob/master/train.py
```

```

#Esta função é muito parecida com a do treino, porém, usa dados de validação p
@torch.no_grad() #Desliga o cálculo dos gradientes, importante em termos compu
def Calc_Loss (modelo, loop_eval = 500):

    output = {}
    modelo.eval() #Desliga modo de treino

    for split in ['train', 'val']: #Calcula a loss para os conjuntos de treino
        losses = torch.zeros (loop_eval) #Cria um tensor com int "0" com tamar

        for i in range (loop_eval): #Por cada loop_eval o modelo vai ser alime

            x, y = Batch (val) #Quando for train: #Qua
            logits = modelo (x) #Alimenta o modelo com dados de treino #A

            B, T, V = logits.shape

            loss = loss_function (logits.view (B * T, V), y.view (B * T))
            losses[i] = loss.item() #Armazena as losses no tensor

        output[split] = losses.mean() #Calcula a média, mais preciso

    modelo.train() #Volto ao modo de treino
    return output

```

Tiny-GPT Treino

Só para ter noção, o modelo base apresentado pelo paper *Attention Is All You Need* foi treinado com 100k steps. :)

```

In [70]: #modelo = TINY_GPT().to(device) #GPU

#optimizer = torch.optim.AdamW (modelo.parameters(), lr = 3e-4) #Usa-se AdamW
#loss_function = nn.CrossEntropyLoss() #Função de perda que mede quão diferent
#Training (modelo, 5000)

```

Geração de Texto ★

Esta função foi altamente influenciada por este rep -> [nanoGPT](#)

```

In [71]: @torch.no_grad() #Desliga o cálculo de gradientes, torna a inferência mais ráp
def Model_Infer (modelo, max_tokens, indice, temperatura = 1, top_tokens = Non

    modelo.eval() #Desliga o treino

    for i in range (max_tokens): #Vai adicionando token a token

```

```

    indice_condition = indice[:, -Sequence_Length:] #Alimento o modelo com o
    logits = modelo (indice_condition) #Aplicação da Arquitetura
    logits = logits[:, -1, :] / temperatura #Foca no que gerou anteriormente

    #https://github.com/karpathy/nanoGPT/blob/master/model.py
    if top_tokens is not None: #TopK funciona como um limitador que ajuda o mc
        V = torch.topk (logits, min(top_tokens, logits.size (-1))) #http
        logits [logits < V[:, [-1]]] = -float('Inf') #Substitui os toke

    probs = F.softmax (logits, dim = -1) #Aplica Softmax para ter as probabili
    prev = torch.multinomial (probs, num_samples = 1) #Seleciona um token
    indice = torch.cat ((indice, prev), dim = 1) #Adiciona esse token ao cont

    return indice

```

Tiny-GPT Inferência

```

In [72]: #contexto = torch.zeros ((1, 1), dtype = torch.long, device = device) #Cria um
#indice = Model_Infer (modelo, max_tokens = 500, indice = contexto)

#print (decode(indice[0].tolist()))

```

CPU vs GPU Perfomance de Treino

Modelo

- Número de Parâmetros -> 250 000
- Batch -> 8
- Sequence_Length -> 64
- Embedding_Dimension -> 64
- Head_Size -> 8
- Number_Head -> 8
- Blocks -> 4

Treino

- Optimizer -> AdamW, lr = 0,0003
- Função de Loss -> CrossEntropyLoss

Dados CPU e GPU

- CPU -> AMD Ryzen 5 4500 6-Core Processor
- GPU -> T4 GPU

```
In [73]: model_CPU = torch.load("Modelo_CPU")
time_training_CPU = model_CPU["time_training"]

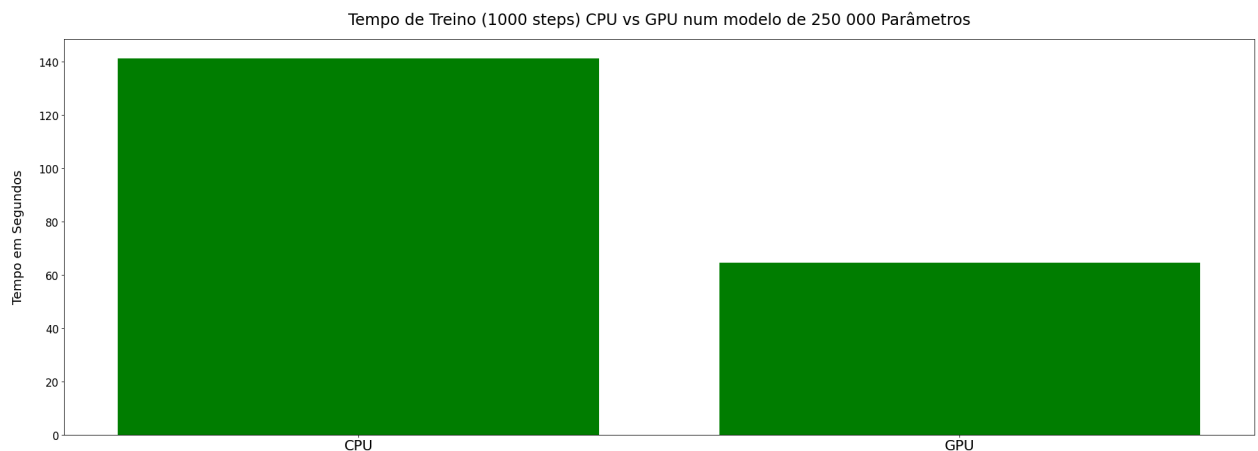
model_GPU = torch.load("Modelo_GPU", map_location = device)
time_training_GPU = model_GPU["time_training"]

pltt = {
    "CPU": time_training_CPU,
    "GPU": time_training_GPU
}
```

```
In [74]: plt.figure (figsize = (30, 10), dpi = 80)
plt.bar (pltt.keys(), pltt.values(), color = "green")

plt.xticks (size = 20)
plt.yticks (size = 15)
plt.ylabel ("Tempo em Segundos", size = 18, labelpad = 20)

plt.title ("Tempo de Treino (1000 steps) CPU vs GPU num modelo de 250 000 Parâmetros")
plt.show()
```



Scaling Laws

Kaplan Scaling Laws

As **Scaling Laws** apresentadas pela OpenAI demonstraram empiricamente que aumentar a escala de treino de um modelo (**número de parâmetros, quantidade de tokens e poder computacional**) tende a torná-lo mais capaz e a melhorar a sua qualidade.

Em outras palavras, modelos maiores tendem a apresentar melhor desempenho, desde que exista também dados e compute suficientes para os treinar adequadamente.

Vamos ver se é verdade..

Discussão

Com este estudo, comprovou-se que é sim verdade.

Aumentar o número de Parâmetros de um Modelo torna-o efetivamente melhor!

```
In [77]: model_500k = torch.load("Modelo_500k", map_location = device)
train_loss_500k = model_500k["train loss"]
val_loss_500k = model_500k["val_loss"]

model_2M = torch.load("Modelo_2M", map_location = device)
train_loss_2M = model_2M["train loss"]
val_loss_2M = model_2M["val_loss"]

print (f"Train Loss 500k -> {train_loss_500k}")
print (f"Train Loss 2M -> {train_loss_2M}\n")

print (f"Val Loss 500k {val_loss_500k}")
print (f"Val loss 2M {val_loss_2M}")
```

```
Train Loss 500k -> [tensor(4.5448), tensor(2.3078), tensor(2.1986), tensor(2.1197), tensor(2.0510)]
Train Loss 2M -> [tensor(4.3285), tensor(2.1819), tensor(2.0208), tensor(1.9246), tensor(1.8526)]
```

```
Val Loss 500k [tensor(4.5451), tensor(2.3037), tensor(2.2029), tensor(2.1201), tensor(2.0492)]
```

```
Val loss 2M [tensor(4.3299), tensor(2.1779), tensor(2.0179), tensor(1.9288), tensor(1.8589)]
```

```
In [78]: plt.figure (figsize = (30, 10), dpi = 80)

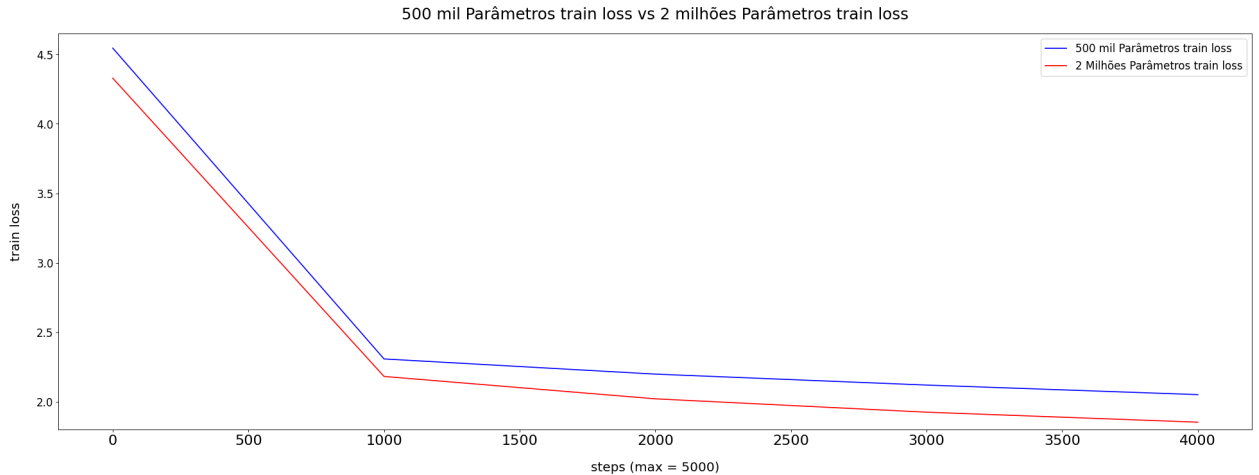
plt.plot (list(range(0, 5000, 1000)), train_loss_500k, color = "blue", label = "500 mil Parâmetros train loss")
plt.plot (list(range(0, 5000, 1000)), train_loss_2M, color = "red", label = "2 Milhões Parâmetros train loss")
#plt.plot (list(range(0, 5000, 1000)), val_loss_500k, color = "green", label = "500 mil Parâmetros val loss")
#plt.plot (list(range(0, 5000, 1000)), val_loss_2M, color = "red", label = "2 Milhões Parâmetros val loss")

plt.xticks (size = 20)
plt.xlabel("steps (max = 5000)", size = 18, labelpad = 20)

plt.ylim(1.8000, 4.6500)
plt.yticks (size = 15)
plt.ylabel ("train loss", size = 18, labelpad = 20)

plt.legend(["500 mil Parâmetros train loss", "2 Milhões Parâmetros train loss"])
plt.title ("500 mil Parâmetros train loss vs 2 milhões Parâmetros train loss",
```

```
plt.show()
```



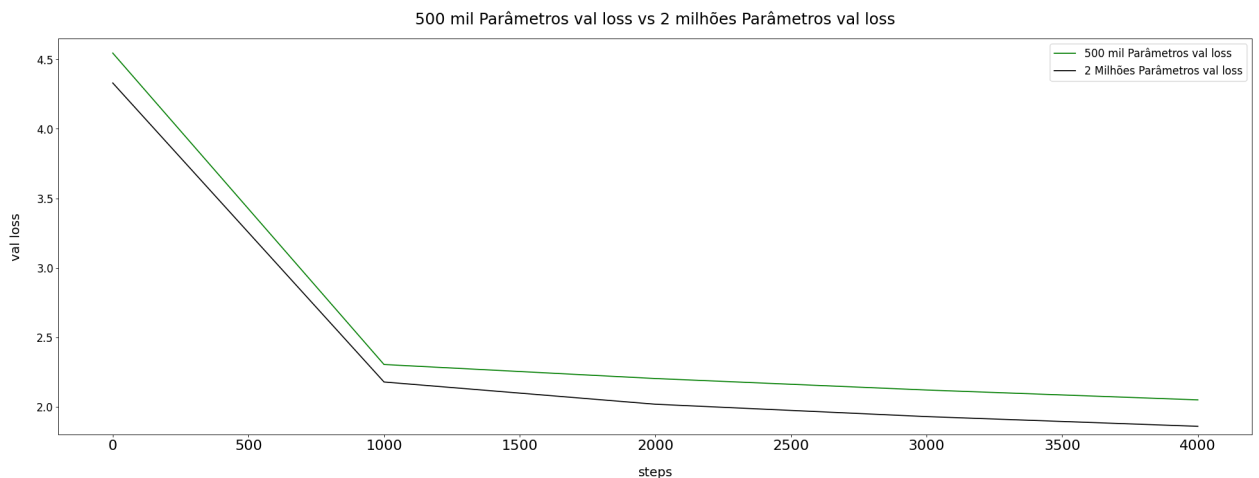
```
In [79]: plt.figure (figsize = (30, 10), dpi = 80)

plt.plot (list(range(0, 5000, 1000)), val_loss_500k, color = "green")
plt.plot (list(range(0, 5000, 1000)), val_loss_2M, color = "black")

plt.xticks (size = 20)
plt.xlabel("steps", size = 18, labelpad = 20)

plt.ylim(1.8000, 4.6500)
plt.yticks (size = 15)
plt.ylabel ("val loss", size = 18, labelpad = 20)

plt.legend(["500 mil Parâmetros val loss", "2 Milhões Parâmetros val loss"], f
plt.title ("500 mil Parâmetros val loss vs 2 milhões Parâmetros val loss", siz
plt.show()
```



Chinchilla Scaling Laws

As **Chinchilla Scaling Laws**, apresentadas pela DeepMind, vieram refinar as Scaling Laws.

Este trabalho demonstrou empiricamente que, para um dado orçamento computacional, existe uma **relação ótima entre o número de parâmetros e a quantidade de tokens de treino**.

Em particular, mostrou-se que muitos modelos anteriores estavam sub-treinados (*undertrained*), pois utilizavam demasiados parâmetros relativamente à quantidade de dados.

Assim, em vez de apenas aumentar o tamanho do modelo, é mais eficiente equilibrar corretamente **Parâmetros e Dados** para maximizar a performance. Fórmula que o paper apresentou:

$$\text{Número de Tokens} \approx 20 * \text{Número de Parâmetros}$$

O primeiro GPT apresentado pela OpenAI em 2018, tinha **120 milhões de Parâmetros** e foi treinado com um corpus de **700 milhões de Tokens**. **De acordo com as Chinchilla Scaling Laws, o modelo estava claramente sub-treinado e a desperdiçar recursos computacionais.**

No meu caso em específico, que temos inicialmente um dataset definido, podemos trocar a fórmula e tentar chegar ao número ideal de parâmetros para o nosso modelo.

$$\text{Número de Parâmetros} \approx \frac{\text{Número de Tokens}}{20}$$

Vamos testar..

Discussão

Este estudo, comprovou que as Chinchilla Scaling Laws fazem sentido! **Aumentar o número de Parâmetros de um Modelo torna-o efetivamente melhor porém, quão melhor e a que custo ?**

Como vimos nos gráficos, um modelo muito maior (3.5M de Parâmetros), realmente obteve valores de training loss e val loss melhores em relação ao modelo mais pequeno (20K Parâmetros) porém, a diferença não é assim tão significativa comparado com as diferenças de tamanho do modelo.

Vale lembrar que treinar um modelo maior, é mais caro e mais dispendioso em termos de tempo!

Recomenda-se usar para produção um modelo superior em relação aos números de parâmetros que a fórmula dá.

A ideia desta Law é como explicar que dentro de um avião, se colocarmos uma

migalha até está tudo bem, porém, em termos de eficiência e combustível não será a melhor opção.

Ao contrário, se para essa migalha usarmos um carro para a transportar, acaba por ficar mais eficiente e torna-se de maneira clara a melhor escolha.

```
In [80]: tokens_count = len(tensor)
print (f"Número de Tokens -> {tokens_count}")
print (f"Número Ideal de Parâmetros -> {tokens_count // 20}")
```

Número de Tokens -> 314636

Número Ideal de Parâmetros -> 15731

```
In [83]: model_20k = torch.load("Modelo_20K", map_location = device)
train_loss_20k = model_20k["train_loss"]
val_loss_20k = model_20k["val_loss"]

model_3_5M = torch.load("Modelo_3.5M", map_location = device)
train_loss_3_5M = model_3_5M["train_loss"]
val_loss_3_5M = model_3_5M["val_loss"]
```

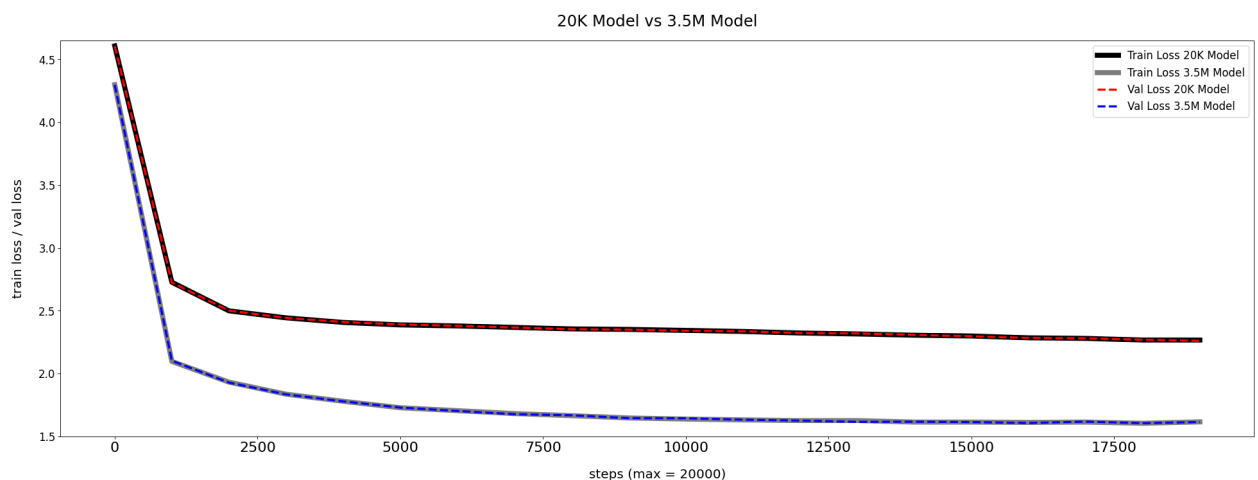
```
In [84]: plt.figure (figsize = (30, 10), dpi = 80)

plt.plot (list(range(0, 20000, 1000)), train_loss_20k, color = "black", lw = 7)
plt.plot (list(range(0, 20000, 1000)), train_loss_3_5M, color = "gray", lw = 7)
plt.plot (list(range(0, 20000, 1000)), val_loss_20k, color = "red", lw = 3, ls = 'dashed')
plt.plot (list(range(0, 20000, 1000)), val_loss_3_5M, color = "blue", lw = 3, ls = 'dashed')

plt.xticks (size = 20)
plt.xlabel("steps (max = 20000)", size = 18, labelpad = 20)

plt.ylim(1.5000, 4.6500)
plt.yticks (size = 15)
plt.ylabel ("train loss / val loss", size = 18, labelpad = 20)

plt.legend(["Train Loss 20K Model", "Train Loss 3.5M Model", "Val Loss 20K Model", "Val Loss 3.5M Model"])
plt.title ("20K Model vs 3.5M Model", size = 22, pad = 20)
plt.show()
```



Eficiência Computacional nos Transformers

Os Transformers são arquiteturas que funcionam principalmente por multiplicação de matrizes.

Por esse motivo, como já vimos, os modelos correm por norma em GPUs por estas apresentarem uma maior capacidade de paralelismo em relação às CPUs.

Um dos pontos mais importantes, é a **eficiência da GPU dado um x modelo**. Ou seja, eu posso ter uma GPU mas qual a **taxa de eficiência da minha GPU em relação ao meu modelo ? Estou a usar toda a capacidade da minha GPU ?**

Para conseguir chegar a um bom resultado, é necessário introduzir dois pontos fundamentais:

- **FLOPs** -> Número de Operações com Pontos Flutuantes realizadas por um modelo.
- **MFU** -> Percentagem dos FLOPs teóricos máximos da GPU que estão efetivamente a ser utilizados pelo modelo durante o treino ou inferência.

É preciso salientar que as GPUs têm uma enorme capacidade teórica mas um modelo raramente atinge a capacidade teórica total. MFU mede essa eficiência real!

FLOPs (Floating Point Operation)

Algumas fórmulas importantes:

$\text{FLOPs por Treino} \approx 6 * \text{Parâmetros Non Embedding} * \text{Tokens}$

$\text{FLOPs por Token} \approx 2 * \text{Parâmetros Non Embedding} * \text{Tokens}$

$\text{FLOPs por Inferência} \approx \text{FLOPs por Token} * \text{Tokens}$

```
In [85]: #Todas as Funções Usadas para calcular estes parâmetros estão presentes neste
#Foram usados os seguintes modelos -> 450.5K Parâmetros vs 3.2M Parâmetros (Pa
FLOPs = {
    "flops_per_treino450.5K": 1.38e9,
    "flops_per_treino3_2M": 9.83e9,

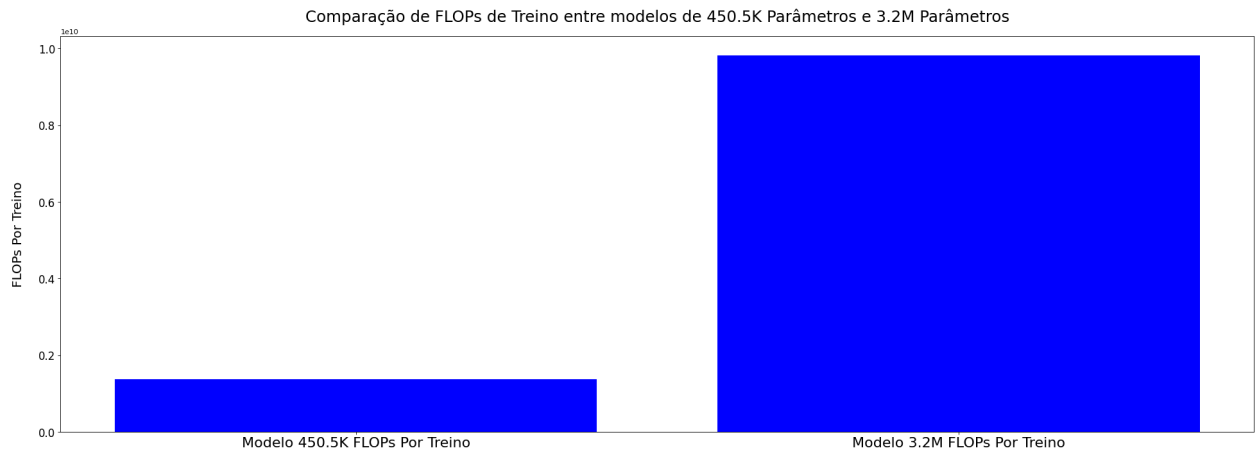
    "flops_per_infer450.5k": 4.61e8,
    "flops_per_infer3_2M": 3.28e9
}
```

```
In [86]: plt.figure (figsize = (30, 10), dpi = 80)

plt.bar (["Modelo 450.5K FLOPs Por Treino", "Modelo 3.2M FLOPs Por Treino"], [
#plt.bar (FLOPs.keys(), FLOPs.values())

plt.xticks (size = 20)
plt.yticks (size = 15)
plt.ylabel ("FLOPs Por Treino", size = 18, labelpad = 20)

plt.title ("Comparação de FLOPs de Treino entre modelos de 450.5K Parâmetros e 3.2M Parâmetros")
plt.show()
```

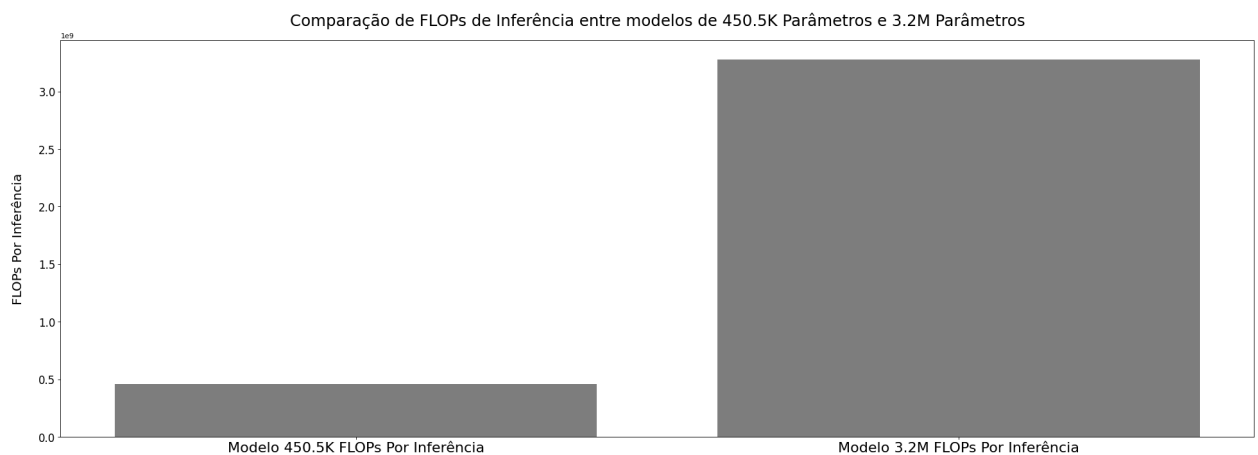


```
In [87]: plt.figure (figsize = (30, 10), dpi = 80)

plt.bar (["Modelo 450.5K FLOPs Por Inferência", "Modelo 3.2M FLOPs Por Inferên
#plt.bar (FLOPs.keys(), FLOPs.values())

plt.xticks (size = 20)
plt.yticks (size = 15)
plt.ylabel ("FLOPs Por Inferência", size = 18, labelpad = 20)

plt.title ("Comparação de FLOPs de Inferência entre modelos de 450.5K Parâmetros e 3.2M Parâmetros")
plt.show()
```



MFUs (Model FLOPs Utilization)

Algumas fórmulas importantes:

$$\frac{\text{FLOPS}}{\text{Achieved Model FLOPS}} \approx \frac{\text{FLOPs}}{\text{segundos para completar todos os FLOPs}}$$

(!NOTA: FLOPs = quantidade de operações, FLOPS = operações por segundo)

$$\text{MFU (\%)} = \frac{\text{FLOPS}}{\text{Peak Hardware FLOPS}}$$

1. Tanto pode ser aplicado para Treino de um modelo ou para Inferência de um modelo!
2. Ao medir os FLOPS é necessário deixar o modelo correr o treino ou inferência durante alguns steps e depois realizar uma divisão para obter um tempo real e confiável.
3. Peak Hardware FLOPS é uma característica do fabricante da GPU! Usa-se sempre com o tipo de dado FP16 que é o tipo de dado usado nos Transformers.
4. Por norma tenta-se atingir cerca de 50/80 % de MFU para considerar uma boa utilização da GPU.

```
In [88]: #https://github.com/Eliezer-Carvalho/Tiny-GPT/blob/main/tools/mfu_count.py
#Vamos realizar o estudo com o modelo de 3.2 Milhões de Parâmetros #E vamos ca
#No meu caso treinei o modelo com a GPU do Colab T4GPU

tempo = 108 #segundos #1000 steps de treino
peak_FLOPS = 65.13e12 #https://www.techpowerup.com/gpu-specs/tesla-t4.c3316

FLOPS = (FLOPs["flops_per_treino3_2M"] * 1000 / tempo) #Porquê 1000 ? Porque f

MFU = FLOPS / peak_FLOPS * 100

print (f"MFU -> {round (MFU, 3)}%")
```

MFU -> 0.14%

Interpretação

O modelo de 3.2 Milhões de Parâmetros Non Embedding está a utilizar apenas 0.14% da capacidade da T4GPU! A GPU está 99,985% ociosa!

Como Escolher uma GPU ?

Na escolha de uma GPU para hospedar um modelo há 3 fatores importantes:

- **FLOPS** -> Já navegámos e já vimos a maneira de calcular os FLOPS de um modelo. Relembro que os FLOPS de um modelo nunca têm a capacidade de atingir o limite de FLOPS de uma GPU. Fica sempre entre 40 a 70%.
- **VRAM** -> É a memória da GPU, é necessário ter em conta porque o tamanho do modelo (Bytes) tem que caber dentro da VRAM.
- **Bandwidth** -> Quantidade de dados que a GPU consegue ler ou escrever na memória por segundo. Afeta a velocidade de treino e inferência.

No meu exemplo, a minha GPU pessoal ([NVIDIA GeForce GTX 1660 SUPER](#)), tinha capacidade de hospedar o modelo de 3.5 Milhões de Parâmetros que treinámos!